CADense CMS Wrapper Studio: Architecture & Implementation Walkthrough

This document outlines the complete architectural blueprint and engineering strategy for **CADense CMS Studio**, a dedicated wrapper application built on top of SolidCraft (v0.23.1+, egui 0.36). It details how to achieve relaxed, rule-free sheet-metal folding and multi-port parametric scaling for cable management system (CMS) products—such as bends, tees, reducers, and risers—while maintaining **100% Zero-Touch Upstream Compliance (Rule R1)**.

1. Architectural Strategy & Rule R1 Compliance

To ensure upstream SolidCraft updates from GitHub can be pulled effortlessly without merge conflicts, CADense follows **Rule R1 (Zero-Touch Upstream Compliance)**:

- **Upstream Untouched:** All core SolidCraft crates (crates/doc, crates/engine, crates/ui-egui, etc.) remain completely unmodified.
- **Dedicated Wrapper Binary (apps/cadense-studio)**: The application runs via cargo run -p cadense-studio. It imports SolidCraft's viewport rendering and document state as path dependencies.
- **Additive Extension Crates (crates/cadense-dsf, crates/cadense-trufold-core, crates/cadense-order)**: All relaxed folding math, 2D polygon slicing, taper-aware scaling, and .dsf container packaging live in isolated workspace crates.

2. UI Layout & Reusing SolidCraft's 3D Viewport

Rather than replacing SolidCraft's interface, the wrapper app **reuses SolidCraft's 3D viewport widget, graphics options, and camera navigation toolbar as-is**, framing it with a custom top-level **CMS Ribbon Toolbar** and side inspector panels.

Key UI Integration Points:

Reused 3D Viewport Component: The center viewport calls SolidCraft's native rendering and camera controller. Users retain all built-in SolidCraft graphics settings (orthographic/perspective, shading, shadows, wireframe overlays), 3D navigational gizmos, and orbit

---

two-stage workflow—**Stage 1: Interactive 2D-to-3D Template Authoring** and **Stage 2: Independent Multi-Port Parametric Scaling**—maps directly onto a clean, two-phase architecture inside your wrapper application (apps/cadense-studio) without modifying any upstream SolidCraft files.

Because your 2D CAM DXF files already include the unfolded sidewalls and edge return flanges in the outer cut contour, the wrapper app does **not** need SolidCraft's sheet.flange generator to invent new geometry—it only needs to **slice the existing 2D flat polygon along user-drawn bend lines**, fold the resulting faces into 3D with a bend radius, and lock vertex groups to independent ports ($P_1 . P_4$) for scaling.

Stage 1: Template Authoring (DXF Import → Bend Lines & Slot Tagging → Live 3D SolidCraft View)

When you open a 2D laser-cutting DXF in your wrapper app, the system converts it into a reusable product template (.dsf / Danish Sheet Fold container) through four steps:

1. Default Sheet Metal Initialization from 2D DXF

- **Contour Stitching:** The DXF importer (dxf_loader) reads LINE, ARC, and LWPOLYLINE entities, heals micro-gaps ($\epsilon = 10^{-3}$ mm), and separates the outer perimeter loop (CUT) from inner closed loops (HOLES).

- **Immediate 3D Flat Blank:** By default, the wrapper reads the user's active sheet metal settings (e.g., Thickness $T = 1.5$ mm, Inner Bend Radius $R = 1.5$ mm, K-factor $K = 0.42$) and sends a base sketch + extrude(T) to the embedded SolidCraft engine. The user immediately sees the flat 3D sheet metal blank in the SolidCraft 3D viewport.

2. Applying Bend Lines (Including Pre-Drawn Edge Flanges)

- **Drawing/Selecting Bend Lines on the 2D Canvas:** When the user draws or snaps a bend line across the 2D drawing and assigns a bend angle (e.g., $+90°$ up) and radius R:

    1. **2D Polygon Slicing (No Tear Reliefs):** Your wrapper's polygon clipper slices the 2D face straight along the bend line segment (offset by half the bend zone width) without running SolidCraft's perpendicular-edge or tear-relief checks.

    2. **Chained Edge Flanges (Base → Sidewall → Return Flange):** Because the 2D DXF already has the return flange drawn outboard of the sidewall, adding a second parallel bend line near the outer edge automatically creates a **2-stage parent-child chain** in the kinematic bend tree:

        $$\text{Base Floor (Root)} \xrightarrow{\text{Bend 1}(+90°, R)} \text{Sidewall} \xrightarrow{\text{Bend 2}(+90°, R)} \text{Edge Return Flange}$$

    3. **Live 3D Conversion in SolidCraft Viewport:** On every bend line addition or edit, the solver recalculates the 3D transform matrix ($\mathbf{M}_{\text{child}} = \mathbf{M}_{\text{parent}} \cdot \mathbf{T} \cdot \mathbf{R} \cdot \mathbf{T}^{-1}$) for each split face and cylindrical bend radius patch, then updates the SolidCraft document in real time so the 3D viewport shows the exact folded part with smooth bend radii.

3. Tagging Coupler Mounting Slots (Rigid Hole Locking)

- **Identifying Edge Slots:** In the 2D canvas, the user selects the obround loops near each port mouth (e.g., OBROUND 20X8 or OBROUND 30X7) and tags them as **Coupler Mounting Holes** bound to a specific Port (P_i).

- **Rigid Anchor Rule:** Instead of scaling the vertices of an obround slot when the tray width or height changes (which would stretch a 20×8 mm bolt slot into an oval), each tagged slot is stored by its **rigid template shape** plus a **fixed setback offset** relative to its parent Port edge and sidewall bend line.

Stage 2: Multi-Port Parametric Scaling (W_i, H_i Per Opening)

Once the bend lines and coupler slots are tagged, the user defines the **Opening Ports** ($P_1 . P_N$) on the template. Every CMS product family follows a specific port cardinality rule:

Product Type	Port Count	Port Dimension Rule	Behavioral Example
Straight, 45°/90° Bend, Inside/Outside Riser	2 Ports (P_1, P_2)	**Coupled ($W_1 = W_2$, $H_1 = H_2$)**	Changing $W = 300$ mm, $H = 100$ mm updates both ends symmetrically across all riser/bend segments.
Reducer (Concentric / Left / Right)	2 Ports (P_1, P_2)	**Independent ($W_1 \neq W_2$, $H_1 \neq H_2$)**	$P_1 = 200 \times 100$ mm, $P_2 = 100 \times 50$ mm. Base tapers in X-Y;

Product Type	Port Count	Port Dimension Rule	Behavioral Example
			sidewalls become 2D trapezoids that fold into sloped 3D ramps.
Unequal Tee	3 Ports (P_1, P_2, P_3)	**Independent (W_1, H_1 | W_2, H_2 | W_3, H_3)**	Main In (P_1), Main Out (P_2), and Branch (P_3) each scale their own mouth width and sidewall height independently.
Unequal Cross	4 Ports (P_1, P_2, P_3, P_4)	**Independent ($W_1 . W_4$, $H_1 . H_4$)**	Four independent opening mouths connected by four chamfered/transitioning corner sidewalls.

How the Parametric Scaling Math Works Without Breaking the 2D/3D Geometry

To allow a Reducer ($P_1 = 200 \times 100$ mm, $P_2 = 100 \times 50$ mm) or an Unequal Tee to change one port's width and height while keeping all other ports intact, each vertex in the 2D template is governed by **three geometric rules**:

1. Independent Width Displacement Along the Port Mouth (ΔW_i)

Each port P_i has an origin center $\mathbf{c}_i$, an outward normal $\hat{\mathbf{n}}_i$, and a lateral width axis $\hat{\mathbf{w}}_i$ in the 2D base floor plane.

- When Port i changes width from $W_{i,\text{base}}$ to $W_{i,\text{new}}$, only the vertices anchored to Port i shift laterally along $\hat{\mathbf{w}}_i$:

$$\Delta\mathbf{p}_{\text{width}}(v) = \alpha_{v,i} \cdot \left(W_{i,\text{new}} - W_{i,\text{base}}\right) \cdot \hat{\mathbf{w}}_i$$

where $\alpha_{v,i} = -0.5$ for the left corner of Port i, $+0.5$ for the right corner of Port i, and 0 for vertices belonging to other ports.

- **Result:** On a Reducer where $W_1 = 200$ mm and $W_2 = 100$ mm, the base floor automatically forms the exact trapezoid connecting the 200 mm mouth to the 100 mm mouth, Rotating the sidewall bend line (Bend 1) to the new taper angle ϕ.

2. Taper-Aware Perpendicular Projection for Sidewall Heights (ΔH_i) & Edge Flanges

This is the step where standard parametric sketches fail: if a sidewall bend line rotates to a taper angle ϕ (because $W_1 \neq W_2$ on a reducer or unequal tee), pushing the sidewall and edge flange vertices along the global X or Y axis skews the flange thickness and breaks the fold.

- Instead, **after** the base floor bend line endpoints ($\mathbf{b}_{\text{start}}, \mathbf{b}_{\text{end}}$) move to their new widths W_1 and W_2, the solver calculates the **live outward unit normal** $\hat{\mathbf{n}}_{\text{edge}} = \left(-\hat{d}_y, \hat{d}_x\right)$ perpendicular to that specific bend line:
    - The start of the sidewall projects outward along $\hat{\mathbf{n}}_{\text{edge}}$ by H_1 (e.g., 100 mm).
    - The end of the sidewall projects outward along $\hat{\mathbf{n}}_{\text{edge}}$ by H_2 (e.g., 50 mm).

    - The **Edge Return Flange** (Bend 2) projects outward perpendicular to the sloped top edge of the sidewall by its constant flange width (e.g., 11 mm or 15 mm).

- **Result:** In 2D, the reducer sidewall becomes a clean trapezoid (100 mm tall at Port 1 tapering to 50 mm tall at Port 2) with a parallel 11 mm return flange along its sloped top edge. When folded $90°$ along Bend 1 and Bend 2, it forms a continuous 3D tapered channel with zero skew.

3. Rigid Coupler Hole Re-Anchoring & Height-Based Slot Rules

- After the sidewall vertices move to their new W_i and H_i positions, the tagged coupler mounting holes are placed back onto the sidewall at their exact local setback from Port i's mouth edge and projected along $\hat{\mathbf{n}}_{\text{edge}}$.
- If the port height crosses your threshold rule (e.g., $H_i = 50$ mm vs. $H_i = 100$ mm), the template can also switch between 1 centered obround slot row and 2 vertically spaced slot rows while keeping each individual obround slot's dimensions (20×8 mm) completely rigid.

End-to-End Wrapper Data Flow (Zero Upstream Changes)

1. **Stage 1 Mode (Template Authoring Tab in Wrapper):**
    - Load 2D CAM .dxf → Set T, R, K → Draw/pick Bend 1 (sidewall) and Bend 2 (edge flange) lines → Tag coupler slots → Assign Port $1..N$ vertex groups → Save as .dsf product template.
2. **Stage 2 Mode (Parametric Configurator / Order Mode in Wrapper):**
    - Load .dsf template → User enters Port dimensions (e.g., Reducer $P_1 = 200 \times 100$, $P_2 = 100 \times 50$; or Bend $P_{1,2} = 300 \times 75$).
    - Wrapper updates the 2D flat pattern (ready for instant CNC laser .dxf export) AND regenerates the 3D folded B-rep in the SolidCraft viewport (ready for 3D inspection, .step export, and 3D hidden-line removal data sheets).

Reusing SolidCraft's native 3D viewport to draw bend lines, snap to vertices/edges, and tag coupler mounting holes directly on the 3D sheet-metal model is completely doable—and you can do it **without modifying a single line of upstream SolidCraft code**.

Because SolidCraft's UI (crates/ui-egui) is built on immediate-mode egui (egui 0.36), the 3D viewport is rendered into a standard egui::Rect inside the central panel, and SolidCraft's document/camera state is accessible to your wrapper app (apps/cadense-studio).

Here are the two clean, Zero-Touch mechanisms you can combine to draw and pick directly on the 3D model inside SolidCraft's viewport.

1. How It Works Without Touching Upstream SolidCraft

When your wrapper app runs SolidCraft's viewport in egui, you have two layers of integration that work together seamlessly:

Layer	How It Works (Zero Upstream Edits)	Best Used For
Layer A: Native SolidCraft 3D	Your wrapper sends standard SolidCraft sketch.* commands on the top planar face of	Storing committed **Bend Lines** (Bend 1

Layer	How It Works (Zero Upstream Edits)	Best Used For
Sketch / Construction Lines	the extruded DXF blank (or creates unconsumed 3D construction sketches). SolidCraft's own GPU renderer draws the lines in true 3D with depth-buffering, edge snapping, and hover highlighting.	for sidewalls, Bend 2 for edge return flanges) directly on the 3D model so they orbit, pan, and zoom natively with the part.
Layer B: egui::Painter Screen-Space 3D Overlay	Immediately after SolidCraft renders its 3D viewport Rect, your wrapper reads the active camera's View-Projection matrix ($\mathbf{M}_{\text{proj}} \cdot \mathbf{M}_{\text{view}}$), projects 3D points on the sheet metal surface to 2D screen pixels, and draws interactive gizmos, rubber-band lines, port badges ($P_1 . P_4$), and slot highlights on a foreground egui::LayerId::Foreground.	Live **rubber-band line drawing**, endpoint snapping indicators, bend angle/radius popups (90° / R1.5), **Coupler Slot tags**, and **Port opening arrows**.

2. Step-by-Step User Workflow Inside the 3D Viewport

Step 1: Open DXF → Auto-Extrude as a 3D Sheet Metal Blank

1. When the user opens a 2D laser-cutting .dxf, your wrapper stitches the outer CUT loop and inner HOLES loops (OBROUND 20X8, OBROUND 30X7, etc.) and immediately runs a native SolidCraft sketch + extrude(thickness) command.
2. SolidCraft's 3D viewport immediately displays the shaded 3D flat sheet metal blank with thickness T (e.g., 1.5 mm), complete with all laser-cut notches, miters, edge flanges, and obround holes.

Step 2: Draw Bend Lines Directly on the 3D Face

When the user activates **"Add Bend Line"** in your wrapper toolbar:

1. **3D Raycasting & Vertex/Edge Snapping:**
    - As the user moves the mouse over the SolidCraft 3D viewport, your wrapper converts the mouse cursor position $(x_{\text{screen}}, y_{\text{screen}})$ into a 3D world ray using the inverse of SolidCraft's viewport camera matrix:

$$\mathbf{r}(t) = \mathbf{o}_{\text{cam}} + t \cdot \hat{\mathbf{d}}_{\text{ray}}$$

    - You intersect $\mathbf{r}(t)$ with the plane of the target 3D sheet metal face (initially $Z = T$, or after folding, the 3D plane of whichever face is under the cursor).
    - Your wrapper snaps the 3D intersection point to the nearest DXF vertex, miter corner, or edge midpoint on that face.
2. **Drawing the Line on the 3D Model:**
    - **Click 1 (Start Point) → Drag → Click 2 (End Point):** While dragging, the egui overlay draws a crisp dashed bend line across the 3D face with a live bend direction arrow and angle/radius badge (e.g., ↑ +90.0° | R = 1.5 mm).
    - **Drawing the Edge Return Flange Bend Line:** Even after the sidewall is folded $90°$ up in 3D, the user can either draw the second bend line (Bend 2) directly on the rotated

3D sidewall face **or** toggle the **"Unfold Preview"** button in the toolbar to click on the flat 3D blank and immediately watch it refold.

Step 3: Instant 3D Folding on Every Bend Line Added

As soon as the second endpoint of a bend line is clicked:

1. Your external relaxed solver (crates/cadense-trufold-core) slices the face polygon along that bend line (zero tear-relief checks, zero perpendicular-edge complaints).
2. It rotates the child face around the bend line by θ with inner radius R and inserts the cylindrical bend radius arc.
3. It updates the SolidCraft 3D document in place. In the very next frame (under 1 ms), SolidCraft's 3D viewport shows the face folded up with its smooth bend radius, and the drawn bend line remains visible along the tangent edge of the bend cylinder.

Step 4: Clicking 3D Obround Holes to Tag Coupler Slots & Ports

Because you have the 3D raycaster running over the SolidCraft viewport:

- **Tagging Coupler Holes:** In **"Tag Coupler Slots"** mode, hovering over any 3D obround cutout on the model highlights its 3D perimeter loop in cobalt blue (#3B82F6). Clicking it tags that loop as a rigid coupler slot (OBROUND 20X8 / 30X7) locked to Port P_i.
- **Assigning Opening Ports ($P_1 . P_4$):** In **"Define Port"** mode, clicking the base floor edge at any opening mouth places a 3D Port Triad (W_i, H_i interactive handles) directly in the 3D viewport.

3. Code Pattern: Hooking SolidCraft's 3D Viewport in Your Wrapper

Here is how your wrapper app (apps/cadense-studio) intercepts mouse input, raycasts onto the 3D sheet metal faces, and draws 3D-projected bend lines over SolidCraft's viewport without modifying upstream crates:

Rust

```
use egui::{Color32, LayerId, Order, Painter, Pos2, Rect, Sense, Stroke};
use nalgebra::{Isometry3, Matrix4, Point3, Vector3, Vector4};


pub struct Viewport3DOverlay {
    pub active_tool: TemplateToolMode,
    pub pending_bend_start_3d: Option<(usize, Point3<f64>)>, // (face_id, 3D point)
    pub hovered_snap_point_3d: Option<(usize, Point3<f64>)>,
}


#[derive(PartialEq, Clone, Copy)]
pub enum TemplateToolMode {
    OrbitNavigate,     // Passes all mouse input directly to SolidCraft 3D camera
    DrawBendLine,      // Snaps to 3D face vertices/edges and draws bend line
    TagCouplerSlot,    // Picks 3D obround hole loops on the model
```

```
    AssignOpeningPort,  // Picks port mouth edges for W_i / H_i parametric scaling
}

impl Viewport3DOverlay {
    /// Projects a 3D world point on the SolidCraft model into 2D egui screen coordinates
    pub fn project_3d_to_screen(
        world_pt: &Point3<f64>,
        view_proj: &Matrix4<f64>,
        viewport_rect: Rect,
    ) -> Option<Pos2> {
        let clip = view_proj * Vector4::new(world_pt.x, world_pt.y, world_pt.z, 1.0);
        if clip.w <= 1e-6 {
            return None; // Behind camera
        }
        let ndc_x = (clip.x / clip.w) as f32;
        let ndc_y = (clip.y / clip.w) as f32;

        // Map NDC [-1, 1] to egui viewport_rect pixels
        let screen_x = viewport_rect.left() + (ndc_x + 1.0) * 0.5 * viewport_rect.width();
        let screen_y = viewport_rect.top() + (1.0 - ndc_y) * 0.5 * viewport_rect.height();

        Some(Pos2::new(screen_x, screen_y))
    }

    /// Unprojects an egui mouse position into a 3D picking ray in SolidCraft world space
    pub fn screen_to_3d_ray(
        mouse_pos: Pos2,
        inv_view_proj: &Matrix4<f64>,
        viewport_rect: Rect,
    ) -> (Point3<f64>, Vector3<f64>) {
        let ndc_x = ((mouse_pos.x - viewport_rect.left()) / viewport_rect.width()) * 2.0 - 1.0;
        let ndc_y = 1.0 - ((mouse_pos.y - viewport_rect.top()) / viewport_rect.height()) * 2.0;

        let near_clip = Vector4::new(ndc_x as f64, ndc_y as f64, -1.0, 1.0);
```

```
    let far_clip  = Vector4::new(ndc_x as f64, ndc_y as f64,  1.0, 1.0);

    let near_world = inv_view_proj * near_clip;
    let far_world  = inv_view_proj * far_clip;

    let p_near = Point3::from_homogeneous(near_world).unwrap();
    let p_far  = Point3::from_homogeneous(far_world).unwrap();
    let dir = (p_far - p_near).normalize();

    (p_near, dir)
  }

  /// Renders interactive 3D bend lines, snap points, and port gizmos over SolidCraft's viewport
  pub fn draw_on_solidcraft_viewport(
    &mut self,
    ctx: &egui::Context,
    viewport_rect: Rect,
    view_proj: &Matrix4<f64>,
    committed_bend_lines_3d: &[(Point3<f64>, Point3<f64>, f64, f64)], // (p0, p1, angle_deg, radius)
  ) {
    let painter = ctx.layer_painter(LayerId::new(Order::Foreground,
egui::Id::new("trufold_3d_overlay")))
        .with_clip_rect(viewport_rect);

    // 1. Draw all committed 3D bend lines directly on the 3D model
    for (p0, p1, angle_deg, radius) in committed_bend_lines_3d {
      if let (Some(s0), Some(s1)) = (
        Self::project_3d_to_screen(p0, view_proj, viewport_rect),
        Self::project_3d_to_screen(p1, view_proj, viewport_rect),
      ) {
        painter.line_segment([s0, s1], Stroke::new(2.5, Color32::from_rgb(59, 130, 246)));
        let mid = s0 + (s1 - s0) * 0.5;
        painter.text(
          mid,
```

```
            egui::Align2::CENTER_BOTTOM,
            format!("{:.0}° R{:.1}", angle_deg, radius),
            egui::FontId::proportional(12.0),
            Color32::WHITE,
          );
        }
      }

      // 2. Draw live rubber-band bend line & vertex snap ring while drawing
      if let Some((_, snap_pt)) = self.hovered_snap_point_3d {
        if let Some(screen_snap) = Self::project_3d_to_screen(&snap_pt, view_proj, viewport_rect) {
          painter.circle_stroke(screen_snap, 6.0, Stroke::new(2.0, Color32::from_rgb(34, 197, 94)));

          if let Some((_, start_pt)) = self.pending_bend_start_3d {
            if let Some(screen_start) = Self::project_3d_to_screen(&start_pt, view_proj, viewport_rect) {
              painter.line_segment(
                [screen_start, screen_snap],
                Stroke::new(2.0, Color32::from_rgb(250, 204, 21)),
              );
            }
          }
        }
      }
    }
  }
}
```

4. Handling "Draw While Flat" vs. "Draw While Folded" in One Viewport

When adding multiple bend lines (for example, Bend 1 for the sidewall and Bend 2 for the edge flange), you can give the user the best of both worlds inside that single SolidCraft 3D viewport:

1. **Live Folded Mode (Default):** Every time the user finishes drawing a bend line on a 3D face, the face immediately folds up in 3D with its bend radius. The user can then click directly on the newly folded 3D sidewall to draw the edge flange bend line (Bend 2), and the flange folds inward $90°$.
2. **Quick "Flatten / Fold" Toggle (Spacebar or Toolbar Toggle):** Because your kinematic bend tree retains both the 2D flat coordinates and the 3D folded transforms for every face, pressing a single toggle button temporarily sets $\theta_i = 0°$ (displaying the flat 3D sheet metal blank with all drawn bend lines overlaid) or $\theta_i = \theta_{\text{target}}$ (displaying the fully folded 3D tray).

Here is the complete architectural and workflow walkthrough document for your wrapper application (**apps/cadense-studio**). It outlines how your app embeds SolidCraft's native 3D viewport and navigational tools verbatim while introducing a dedicated CMS Ribbon Toolbar and external relaxed folding engine—maintaining 100% Zero-Touch Upstream Compliance (Rule R1).

CADense CMS Wrapper Studio: Architecture & Workflow Walkthrough

This document outlines the architecture, layout hierarchy, and two-stage user workflow for **CADense CMS Studio**, a dedicated wrapper application built on top of SolidCraft (v0.23.1+, egui 0.36) for designing and scaling sheet-metal cable management systems (cable trays, trunking, bends, tees, reducers, and risers).

1. Architectural Strategy & Rule R1 Compliance

To ensure upstream SolidCraft updates from GitHub can be pulled effortlessly without merge conflicts, CADense follows **Rule R1 (Zero-Touch Upstream Compliance)**:

- **Upstream Untouched:** All core SolidCraft crates (crates/doc, crates/engine, crates/ui-egui, etc.) remain completely unmodified.
- **Dedicated Wrapper Binary (apps/cadense-studio)**: The application runs via cargo run -p cadense-studio. It imports SolidCraft's viewport rendering and document state as path dependencies.
- **Additive Extension Crates (crates/cadense-dsf, crates/cadense-trufold-core, crates/cadense-order)**: All relaxed folding math, 2D polygon slicing, taper-aware scaling, and .dsf container packaging live in isolated workspace crates.

2. UI Layout: Reusing SolidCraft's Viewport + Custom CMS Ribbon

Rather than replacing SolidCraft's interface, the wrapper app **reuses SolidCraft's 3D viewport widget, graphics options, and camera navigation toolbar as-is**, framing it with a custom top-level **CMS Ribbon Toolbar** and side inspector panels.

+--+ | [CADense Logo] [File] [Stage 1: Template Authoring] [Stage 2: Parametric Scale] ... | <- Custom CMS Ribbon +--+ | [Tool: Orbit] [Add Bend] [Tag Slot] [Define Port] [Flat/Fold Toggle] [CNC DXF]| <- CMS Action Bar +---+-------------------------------------+ | | [Inspector / Parameter Panel] | | | | | | - Port P1: W=300 mm, H=100 mm | | | - Port P2: W=100 mm, H=50 mm | | SolidCraft 3D Viewport | - Thickness (T): 1.5 mm | | (Reused Native egui Viewport & | - Inner Radius (R): 1.5 mm | | Graphics / Navigation Gizmos) | - K-Factor: 0.42 (HDG) | | | | | | [Apply Parametric Scaling] | | | | | | [Export .DSF / .STEP / .DXF] | +---+-------------------------------------+ | Status: Template 'BN_CT_RED_200x100' Loaded | Bends: 2 Active | Faces: 5 | OK | <- Status Bar +--------------------------

```
----------------------------------------------------------+
```

```
+-------------------------------------------------------------------------------+
ogo]  [File]  [Stage 1: Template Authoring]  [Stage 2: Parametric Scale]  ... | <- C
+-------------------------------------------------------------------------------+
bit ] [ Add Bend ] [ Tag Slot ] [ Define Port ] [ Flat/Fold Toggle ] [ CNC DXF ]| <-
+-------------------------------------+-----------------------------------+
                                      | [Inspector / Parameter Panel]     |
                                      |                                   |
                                      | - Port P1: W=300 mm, H=100 mm     |
                                      | - Port P2: W=100 mm, H=50 mm      |
  SolidCraft 3D Viewport              | - Thickness (T): 1.5 mm           |
Reused Native egui Viewport &         | - Inner Radius (R): 1.5 mm        |
 Graphics / Navigation Gizmos)        | - K-Factor: 0.42 (HDG)            |
                                      |                                   |
                                      | [ Apply Parametric Scaling ]      |
                                      |                                   |
                                      | [ Export .DSF / .STEP / .DXF ]    |
+-------------------------------------+-----------------------------------+
mplate 'BN_CT_RED_200x100' Loaded | Bends: 2 Active | Faces: 5 | OK     | <- Status
+-------------------------------------------------------------------------------+
```

Key UI Integration Points:

1. **Reused 3D Viewport Component:** The center viewport calls SolidCraft's native rendering and camera controller. Users retain all built-in SolidCraft graphics settings (orthographic/perspective, shading, shadows, wireframe overlays), 3D navigational gizmos, and orbit/pan/zoom behaviors.
2. **Custom CMS Ribbon Toolbar:** Placed directly above SolidCraft's viewport, housing domain-specific tools for cable tray and trunking authoring.
3. **Docking Inspector Panels (egui::SidePanel):** Floating or docked side panels manage port dimensions (W_i, H_i), material gauges, K-factors, and template metadata.

3. Stage 1: Interactive 2D-to-3D Template Authoring

When the user opens a laser-cutting 2D CAM DXF file inside the wrapper, they author a reusable product template through the following sequence:

1. **DXF Import & Auto-Blank Generation:**
   - The DXF loader stitches LINE, ARC, and LWPOLYLINE entities ($\epsilon = 10^{-3}$ mm tolerance), separating the outer contour (CUT) from inner slots (HOLES).
   - The app immediately executes a standard SolidCraft sketch + extrude(T) using the user's active thickness setting, displaying the flat 3D blank in SolidCraft's viewport.
2. **Drawing Bend Lines & Edge Flanges directly in 3D:**
   - The user selects the **Add Bend Line** tool on the CMS ribbon.
   - Hovering over the 3D sheet metal model raycasts onto the active face, snapping to vertices, miter corners, or edge midpoints.
   - Clicking start and end points draws a 3D bend line overlay with angle and radius badges (e.g., +90° | R1.5).

- **Relaxed Slicing Engine:** The external solver splits the 2D polygon along the bend line without enforcing SolidCraft's strict tear-relief or perpendicular-edge checks. Pre-drawn edge return flanges (e.g., 11 mm tray lips) automatically form a parent-child chain (Base -> Sidewall -> Return Flange).
- **Live 3D Folding:** Upon completing the line, the solver computes the 3D transform matrices ($\mathbf{M}_{\text{child}} = \mathbf{M}_{\text{parent}} \cdot \mathbf{T} \cdot \mathbf{R} \cdot \mathbf{T}^{-1}$) and inserts cylindrical bend radius patches. SolidCraft's viewport updates instantly to show the folded part.

3. **Tagging Coupler Mounting Slots:**
   - In **Tag Slot** mode, clicking obround cutouts (OBROUND 20X8, OBROUND 30X7) locks them as rigid hole geometry bound to adjacent port mouths.

4. Stage 2: Independent Multi-Port Parametric Scaling

Once a template is authored, Stage 2 enables instant parametric resizing across different product variations without redrawing:

- **Reducers ($P_1 \neq P_2$):** One port can be set to 200×100 mm and the other to 100×50 mm. The base floor automatically scales into a trapezoid, and sidewalls taper cleanly.
- **Unequal Tees (P_1, P_2, P_3):** Main In, Main Out, and Branch dimensions adjust independently.
- **Bends & Risers ($P_1 = P_2$):** Symmetric opening ports scale uniformly while preserving all miter angles (e.g., 45° or 90°).
- **Taper-Aware Flange Normals ($\hat{\mathbf{n}}_{\text{edge}}$):** Sidewalls and edge return flanges project along live perpendicular unit normals to prevent skew on tapered transitions.

5. File Deliverable Summary

The complete architecture is structured around the following workspace workspace crates:

- **apps/cadense-studio**: The wrapper binary containing the CMS Ribbon, viewport raycaster, and egui UI overlays.
- **crates/cadense-trufold-core**: The relaxed 2D polygon slicer, kinematic bend tree resolver, and taper-aware scaler.
- **crates/cadense-dsf**: The atomic .dsf container reader/writer packaging design.solidcraft and metadata.
- **crates/cadense-order**: CSV order queue batch processor matching the 65-variation Bonn Metal catalog.